Приднестровский государственный университет им. Т.Г. Шевченко
Физико-технический институт

ПРАКТИЧЕСКИЙ КУРС ВЕБ РАЗРАБОТКИ REACT TYPESCRIPT NODE И SUPABASE

Лабораторный практикум

Разработал:
ст. преподаватель
кафедры ИТ
Бричаг Д.В.

г. Тирасполь
2026 г.

Лабораторная работа 2

Страницы приложения и навигация

Продолжите приложение из первой лабораторной. Вместо пустого раздела вы создадите список записей и отдельную страницу подробностей. Пользователь сможет открыть карточку по ссылке, перейти к экрану создания и вернуться к списку. Все страницы будут использовать общее оформление.

Цель работы

Собрать интерфейс SPA из компонентов, описать данные и props с помощью TypeScript и связать страницы через React Router. В результате адрес в браузере будет определять, какую страницу и какую запись видит пользователь.

Обязательный результат

Не менее пяти вымышленных записей своей темы, список, карточка по id и страница создания с понятной заглушкой. На всех экранах есть общий layout и навигация. Неизвестный адрес и отсутствующая запись обрабатываются отдельно. Проект собирается с проверкой типов и остаётся в репозитории ЛР 1.

В этой работе данные находятся в типизированном массиве. Создание, редактирование и удаление записей через формы появятся в ЛР 3. HTTP и Supabase пока не подключаются. Смена темы или создание нового Vite-проекта не требуется.

Теоретическая справка

Компоненты и props

Страница — компонент, соответствующий определённому адресу. Повторяющийся элемент интерфейса удобно выделить отдельно: например, TaskCard показывает одну задачу, а TasksPage выводит несколько таких карточек. Общий layout содержит название приложения, навигацию и место для текущей страницы.

Props — данные, которые родитель передаёт компоненту. Тип props задаёт контракт: TaskCard получает объект Task, а не набор неописанных значений. Компонент читает props и отображает их. Исходные записи хранятся в одном модуле, чтобы список и подробности не расходились.

Метод map превращает массив записей в массив элементов интерфейса. React нужен стабильный key для каждого элемента списка. Используйте id записи: индекс массива и случайное число не обозначают устойчивую идентичность записи.

Маршруты и адрес записи

В SPA переход между внутренними страницами обрабатывается в браузере. React Router сопоставляет адрес с компонентом. BrowserRouter подключается один раз, Routes содержит маршруты, а Route связывает путь и страницу. Здесь используется декларативный режим, без серверного рендеринга и загрузчиков данных.

Link выполняет внутренний переход, NavLink дополнительно позволяет выделить активный пункт. Outlet показывает дочернюю страницу внутри общего layout. Параметр :id в пути обозначает изменяемую часть адреса; useParams читает её значение.

Путь /tasks/t1 означает карточку записи t1. Путь /tasks/new — отдельный экран создания. Адрес /unknown не совпадает с известными маршрутами, а /tasks/missing совпадает с маршрутом карточки, но запись может отсутствовать. Эти ситуации требуют разных сообщений.

Практическая часть

Шаг 1 Продолжите проект первой лабораторной

Откройте основной репозиторий и проверьте, что изменения ЛР 1 сохранены. В корне выполните git status. Не удаляйте историю и не создавайте отдельную папку lab2. Перейдите в frontend и убедитесь, что исходный проект собирается.

```
cd frontend
npm ci
npm run build
npm install react-router
```

Примеры используют пакет react-router и его декларативный API. Все импорты маршрутизатора ниже относятся к этому пакету. Не смешивайте их с инструкциями для других режимов или старых версий. Зафиксируйте установленную версию в package-lock.json; новые зависимости входят в существующий frontend.

Шаг 2 Наметьте страницы своей темы

Выделите три основных адреса: список, подробности и создание. Дополнительно предусмотрите переход с главной страницы к списку и маршрут неизвестной страницы. В примерах: / ведёт к /tasks, /tasks показывает список, /tasks/:id — подробности, /tasks/new — заглушку создания.

Для других тем замените ресурс и названия: issues, transactions, media, games, events, habits, assignments, books или contacts. Например, библиотека использует /books, /books/:id и /books/new. На странице создания явно напишите, что форма появится в следующей работе; не имитируйте сохранение кнопкой без действия.

Шаг 3 Опишите данные приложения

Создайте frontend/src/types/task.ts или соответствующий файл своей темы. Помимо id включите 5–8 содержательных полей из программы курса. Для значений с ограниченным набором используйте строковое объединение. Пример для задач:

```
export type TaskStatus = 'todo' | 'in_progress' | 'done'

export type Task = {
  id: string
  title: string
  description: string
  status: TaskStatus
  dueDate: string
  priority: 'low' | 'medium' | 'high'
  tag: string
}
```

Дата пока хранится строкой YYYY-MM-DD. id также строковый, чтобы напрямую сравнивать его с параметром URL. TypeScript проверяет форму объекта, но не доказывает, что любая строка является корректной датой.

Создайте frontend/src/data/tasks.ts. В этой работе тег — текстовое значение, а база и связи появятся позже. Внесите минимум пять записей с уникальными id, разными статусами и содержательными описаниями. Ниже показана одна запись; ещё четыре подготовьте самостоятельно.

```
import type { Task } from '../types/task'

export const tasks: Task[] = [
  {
    id: 't1',
    title: 'Подготовить план проекта',
    description: 'Описать основной сценарий приложения.',
    status: 'todo',
    dueDate: '2026-10-01',
    priority: 'high',
    tag: 'Учёба',
  },
]
```

Для библиотеки задайте книгу, автора, жанр, статус, оценку и заметку; для расходов — описание, тип операции, сумму, дату и категорию; для мероприятий

— название, описание, начало, окончание, место и категорию. Следуйте минимальному варианту своей темы из программы курса. Типы импортируйте через import type, чтобы сохранить настройки TypeScript из ЛР 1.

Шаг 4 Выделите повторяющуюся карточку

Создайте frontend/src/components/TaskCard.tsx. Компонент должен получать запись через типизированные props и показывать краткое содержание. Полный набор полей будет на странице подробностей.

```
import { Link } from 'react-router'
import type { Task } from '../types/task'

type TaskCardProps = { task: Task }

export function TaskCard({ task }: TaskCardProps) {
  return (
    <article className="task-card">
      <h2>
        <Link to={`/tasks/${task.id}`}>{task.title}</Link>
      </h2>
      <p>{task.description}</p>
      <p>Срок: {task.dueDate}</p>
    </article>
  )
}
```

Имя React-компонента начинается с заглавной буквы. В JSX используется уже переданный task, поэтому компоненту не нужно самостоятельно искать запись в массиве. Добавьте краткое отображение статуса и приоритета; технические значения todo и high замените понятными русскими подписями.

Шаг 5 Соберите страницу списка

Создайте frontend/src/pages/TasksPage.tsx. Список импортирует единый массив данных и передаёт каждую запись в TaskCard. Если записей нет, выводится сообщение; эта проверка пригодится при переходе к реальным данным.

```
import { Link } from 'react-router'
import { TaskCard } from '../components/TaskCard'
import { tasks } from '../data/tasks'

export function TasksPage() {
  return (
    <section>
      <h1>Мои задачи</h1>
      <Link to="/tasks/new">Добавить задачу</Link>
      {tasks.length === 0 ? (
        <p>Задач пока нет.</p>
      ) : (
        <div className="task-list">
          {tasks.map((task) => (
            <TaskCard key={task.id} task={task} />
          ))}
        </div>
      )}
    </section>
  )
}
```

Не копируйте разметку пяти карточек вручную. key задаётся там, где создаётся массив элементов. Это служебное свойство React: если компоненту нужен идентификатор, он получает его как task.id. Поиск и фильтры сейчас не требуются.

Шаг 6 Добавьте страницу подробностей

Создайте frontend/src/pages/TaskDetailsPage.tsx. Определяйте запись по параметру URL, чтобы ссылка работала при прямом открытии и перезагрузке. Не передавайте выбранную запись только через состояние навигации.

```
import { Link, useParams } from 'react-router'
import { tasks } from '../data/tasks'

export function TaskDetailsPage() {
  const { id } = useParams()
  const task = tasks.find((item) => item.id === id)

  if (!task) {
    return (
      <section>
        <h1>Задача не найдена</h1>
        <Link to="/tasks">К списку задач</Link>
      </section>
    )
  }

  return (
    <section>
      <h1>{task.title}</h1>
      <p>{task.description}</p>
      <p>Срок: {task.dueDate}</p>
      <Link to="/tasks">К списку задач</Link>
    </section>
  )
}
```

Дополните подробности остальными содержательными полями своей сущности. Проверка !task должна находиться до обращения к task.title. Для возврата используйте явный адрес списка: команда «назад» по истории браузера может увести за пределы приложения, если карточку открыли напрямую.

Шаг 7 Создайте общий layout и оставшиеся страницы

Создайте frontend/src/app/AppLayout.tsx. Перенесите сюда название и общую оболочку из ЛР 1. Оно должно оставаться одинаковым на списке, карточке, создании и неизвестной странице.

```
import { NavLink, Outlet } from 'react-router'

export function AppLayout() {
  return (
    <div className="app">
      <header>
        <p className="app-title">Task Tracker</p>
        <nav aria-label="Основная навигация">
          <NavLink to="/tasks" end>Задачи</NavLink>
          <NavLink to="/tasks/new">Создать</NavLink>
        </nav>
      </header>
      <main><Outlet /></main>
    </div>
  )
}
```

end означает точное совпадение для пункта списка: он не выделяется на странице /tasks/new. Имя приложения находится в обычном текстовом элементе, а h1 принадлежит текущей странице. Outlet — место, где появится дочерний экран; без него видна только шапка.

Самостоятельно создайте два компонента с именованными экспортами. NewTaskPage в pages/NewTaskPage.tsx содержит h1 «Создание задачи», пояснение о будущей форме и Link к списку. NotFoundPage в pages/NotFoundPage.tsx содержит h1 «Страница не найдена», короткое пояснение и Link к списку. На этом этапе видимые сообщения важнее формы без работающего сценария.

Шаг 8 Подключите маршруты

В frontend/src/main.tsx добавьте импорт BrowserRouter из react-router и оберните существующий App внутри StrictMode. Сохраните импорты CSS и вызов createRoot из первой работы. Ниже показан только фрагмент внутри render:

```
<StrictMode>
  <BrowserRouter>
    <App />
  </BrowserRouter>
</StrictMode>
```

Не добавляйте второй BrowserRouter внутри App. Когда все компоненты страниц созданы, замените frontend/src/App.tsx следующей схемой маршрутов. Имена импортов должны совпадать с экспортами соответствующих файлов.

```
import { Navigate, Route, Routes } from 'react-router'
import { AppLayout } from './app/AppLayout'
import { TasksPage } from './pages/TasksPage'
import { TaskDetailsPage } from './pages/TaskDetailsPage'
import { NewTaskPage } from './pages/NewTaskPage'
import { NotFoundPage } from './pages/NotFoundPage'
import './App.css'

export default function App() {
  return (
    <Routes>
      <Route element={<AppLayout />}>
        <Route index element={<Navigate to="/tasks" replace />} />
        <Route path="tasks" element={<TasksPage />} />
        <Route path="tasks/new" element={<NewTaskPage />} />
        <Route path="tasks/:id" element={<TaskDetailsPage />} />
        <Route path="*" element={<NotFoundPage />} />
      </Route>
    </Routes>
  )
}
```

Маршрут без path задаёт общую оболочку. replace при переходе с / заменяет текущую запись истории, поэтому кнопка браузера «назад» не возвращает на промежуточное перенаправление. Статический путь tasks/new выбирается отдельно от tasks/:id. Для всех внутренних переходов используйте Link или NavLink, а не перезагрузку через location.href.

Шаг 9 Оформите список и навигацию

Сохраните базовые стили ЛР 1 и дополните App.css. Сделайте навигацию заметной, выделите активную ссылку, оставьте видимый фокус клавиатуры. Карточки должны читаться на ноутбуке и узком экране. Пример нескольких правил:

```
nav { display: flex; flex-wrap: wrap; gap: 16px; }
nav a.active { font-weight: bold; }
.task-list { display: grid; gap: 16px; margin-top: 24px; }
.task-card { border: 1px solid #ccc; padding: 16px; }
.task-card h2 { margin-top: 0; }
.task-card { overflow-wrap: anywhere; }
@media (min-width: 760px) {
  .task-list { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
```

Проверьте ширину около 360 пикселей в инструментах браузера: ссылки переносятся, длинный заголовок не создаёт горизонтальную прокрутку. Пройдите навигацию клавишей Tab и откройте ссылку через Enter. Не убирайте outline без равноценной видимой замены.

Шаг 10 Проверьте приложение и сохраните этап

Запустите npm run dev из frontend. Выполните сценарии ниже. Для проверки неизвестной записи используйте id, которого действительно нет в вашем массиве. Номер порта берите из терминала.

1. Откройте / и убедитесь, что показан список своей темы.
2. Откройте две разные карточки и сравните их содержимое.
3. Вставьте адрес существующей карточки в новую вкладку и обновите страницу.
4. Перейдите к созданию: видны заглушка и общая навигация.
5. Откройте /unknown: видна неизвестная страница.
6. Откройте /tasks/missing: видно сообщение об отсутствующей записи.
7. Временно замените массив на пустой и проверьте сообщение списка; затем верните пять записей.
8. Проверьте клавиши «назад» и «вперёд», узкий экран и переходы через Tab.

В Network браузера очистите список запросов и несколько раз перейдите между экранами. При обычном клике по внутренней ссылке не должно быть новой загрузки HTML-документа. Загрузка модулей во время разработки возможна; она не означает полную перезагрузку страницы.

Завершите сервер через Ctrl+C, выполните npm run build и npm run preview. Повторите прямое открытие карточки в preview. Консоль не должна содержать ошибок приложения или предупреждений о key. Публичная настройка возврата index.html для SPA понадобится при размещении в ЛР 10.

Обновите README: добавьте адреса страниц и укажите, что записи демонстрационные. В docs/progress.md добавьте ЛР 2, краткое описание результата и шаги проверки. После успешной сборки вернитесь из frontend в корень и сохраните изменения:

```
cd ..
git status
git add .
git diff --cached --stat
git commit -m "Lab 2: add pages and navigation"
git push
```

Проверьте, что на сервер отправлены новый package-lock.json и все импортируемые файлы. В чистой копии репозитория выполните npm ci и npm run build из frontend. Исходники и история ЛР 1 должны остаться доступны.

Частые проблемы

Link или useParams сообщает об отсутствии Router. Проверьте единственный BrowserRouter в main.tsx. Если видна только шапка, проверьте Outlet и вложенность Route в App.

Карточка не находится по существующему адресу. Сравните имя параметра :id и переменную id из useParams. Значение URL строковое; в примере id массива тоже строковый. Не заменяйте поиск обращением к элементу массива по индексу.

TypeScript требует type-only import. Используйте import type для Task и типов props. Не отключайте strict и не заменяйте типы на any, чтобы скрыть ошибку.

При переходе загружается вся страница. Проверьте, что внутренняя ссылка создана через Link или NavLink. Если новая страница не собирается, сопоставьте имена экспортов, импортов и регистр букв в путях.

Использование ИИ при выполнении работы

Можно попросить ИИ объяснить вложенные маршруты, помочь описать тип вашей сущности или разобрать конкретную ошибку. Например: «Почему мой

layout отображается, а страница внутри него нет?» Приложите относящиеся к ошибке файлы и проверьте ответ по работающему приложению. Использование ИИ необязательно; вы должны уметь объяснить выбор записи по URL и передачу props.

Результаты работы и сдача

В существующем Git-репозитории находятся типы и демонстрационные данные, повторно используемый компонент карточки, страницы списка, подробностей, создания и неизвестного адреса, общий layout, маршруты, стили и обновлённые инструкции. Зависимости и lock-файл сохранены; node_modules и dist исключены.

Предоставьте ссылку на репозиторий и коммит ЛР 2. На демонстрации покажите пять записей, две разные карточки, прямое открытие одной из них, заглушку создания и обе ситуации отсутствия страницы/записи. Объясните связь Route, Outlet и useParams и покажите успешную сборку.

Критерии оценки

Максимум — 10 баллов. Обязательный результат должен быть выполнен независимо от дополнительных визуальных улучшений.

Функционал — 5 баллов: список минимум из пяти записей и повторно используемая карточка (2); подробности выбранной по URL записи со всеми полями (1); экран создания и переходы SPA (1); общий layout и типизированные props (1).

Ошибки и крайние случаи — 2 балла: отдельные сообщения для неизвестного пути и отсутствующей записи (1); прямое открытие и обновление карточки, пустой список, читаемый узкий экран и отсутствие ошибок приложения (1).

Непрерывность проекта — 1 балл: сохранены тема, репозиторий и рабочая сборка ЛР 1, начальный экран развивается в интерфейс того же приложения.

Git и воспроизводимость — 2 балла: актуальные исходники, зависимости и lock-файл доступны преподавателю (1); README и запись этапа позволяют повторить проверку, чистая копия устанавливается и собирается (1).

Вопросы для защиты

1. Чем страница отличается от повторно используемого компонента?
2. Что получает TaskCard через props и зачем здесь тип Task?
3. Почему key должен быть стабильным и где он задаётся?
4. Какую роль выполняют BrowserRouter, Routes и Outlet?
5. Откуда берётся id и почему карточка работает после обновления?
6. Чем отличаются неизвестный путь и отсутствующая запись?
7. Что вы добавите в существующую страницу создания в ЛР 3?

Материалы для самостоятельной работы

React Router — подключение в декларативном режиме: https://reactrouter.com/start/declarative/installation

React Router — вложенные маршруты и параметры: https://reactrouter.com/start/declarative/routing

React — отображение массивов и ключи: https://react.dev/learn/rendering-lists

React — передача props: https://react.dev/learn/passing-props-to-a-component